Optimized Translation of MATLAB to RISC-V Architecture for Brain-Computer Interfaces: Midterm Progress Report

Abstract

Brain-Computer Interfaces (BCIs) represent an important transformative frontier in neurotechnology, offering profound therapeutic avenues for severe neurological disorders such as epilepsy, stroke, and spinal cord injuries. The deployment of these devices in a chronic, fully implantable, edge compute setting imposes severe limitations on power dissipation to prevent irreversible thermal damage to surrounding cortical tissue. Consequently, BCI hardware requires highly specialized microarchitectures that balance real-time, high-throughput signal processing with ultra-low power consumption. The Hardware Architecture for LOw Power BCIs (HALO) addresses these constraints through a design combining application-specific hardware accelerators with a low-power RISC-V microcontroller equipped with the RISC-V Vector (RVV) extension. Despite these critical hardware advancements, a significant toolchain disparity persists between the high-level computational environments preferred by neuroscientists and the low-level execution constraints of the embedded hardware. Algorithm development typically occurs within dynamically typed, high-level frameworks such as MATLAB, whereas the target HALO hardware requires statically typed, meticulously optimized, and explicitly vectorized C code. This report details the ongoing development and preliminary benchmarking of an automated, end-to-end compilation pipeline engineered to bridge this divide. By integrating a TypeScript-based source-to-source compiler with an RVV-optimized C matrix library, the proposed pipeline automatically translates MATLAB algorithms into power-efficient RISC-V binaries. This midterm progress report delineates the architectural design of the translation pipeline, evaluates extensive vectorization benchmarks simulated on the Spike instruction set architecture simulator, resolves anomalies in execution cycle metrics, discusses ongoing debugging efforts concerning determinant computations, and presents a comprehensive strategic roadmap for the remainder of the research term.

Introduction

The intersection of computational neuroscience and embedded systems engineering has catalyzed the development of Brain-Computer Interfaces capable of direct, real-time communication between biological neural networks and digital processing units. Such systems are increasingly central to closed-loop therapeutic applications, where the device must continuously ingest high-density, multi-channel neural recordings, execute complex spectral and statistical feature extraction algorithms, and determine appropriate electrical stimulation parameters to mitigate neurological events such as epileptic seizures.[2] To maximize patient

safety, minimize surgical complications, and ensure device longevity, fully implantable BCIs must adhere to extreme physical and thermal constraints. The most critical of these parameters is the thermal dissipation limit. To prevent irreversible necrosis of the surrounding brain tissue, the surface temperature of the implant must not increase by more than a single degree Celsius, strictly capping the total device power budget at approximately fifteen milliwatts.[3]

The HALO BCI project was conceived and engineered specifically to satisfy these dual, often conflicting demands of real-time, high-throughput neural signal processing and ultra-low power consumption. The HALO architecture deliberately departs from traditional monolithic Application-Specific Integrated Circuit (ASIC) designs, which lack the flexibility to adapt to evolving algorithmic advancements, by employing a modular array of hardware Processing Elements (PEs) orchestrated by a low-power RISC-V microcontroller.[2] This central microcontroller crucially utilizes the RISC-V Vector extension to execute data-parallel signal processing workloads with exceptional efficiency, thereby minimizing instruction fetch overhead and enabling the processor to rapidly transition into low-power sleep states.

However, the clinical and research efficacy of the HALO platform is currently bottlenecked by a profound toolchain disparity. Neuroscientists and algorithmic researchers predominantly utilize MATLAB for its intuitive matrix-oriented syntax, robust signal processing toolboxes, and dynamic typing capabilities, which facilitate the rapid prototyping of complex neural decoding models. Conversely, the HALO architecture's resource-constrained environment strictly demands deterministic, statically typed C code that explicitly leverages RVV intrinsics for parallel execution. The manual translation of advanced, multi-stage signal processing algorithms—such as Fast Fourier Transforms, spectral filtering, and complex matrix decompositions—from MATLAB to vectorized C is an exceedingly error-prone and labor-intensive process that obstructs the rapid deployment of novel clinical therapies.

This research addresses this critical toolchain gap by engineering an automated compilation pipeline that seamlessly translates standard MATLAB scripts into RVV-accelerated C binaries optimized for the HALO platform. The proposed solution relies on the integration, refinement, and expansion of two software modules previously developed in isolation: a source-to-source compiler capable of parsing MATLAB syntax into equivalent C code, and a computational C library that maps standard linear algebra operations directly to RVV assembly instructions.

Significant achievements have been realized during the initial phases of this project. The underlying vectorized matrix library has been successfully replicated and integrated into a unified benchmarking environment utilizing the official RISC-V instruction set architecture simulator. Extensive performance evaluations have been conducted, revealing substantial reductions in execution cycles driven by updated cross-compiler toolchains. Furthermore, critical anomalies in the execution ratios of complex equality checks have been successfully isolated and diagnosed as test-harness artifacts rather than vectorization regressions. In addition, debugging efforts have identified fundamental algorithmic limitations in the current

translation of matrix determinant computations. This progress report establishes the foundational background, outlines the structural design of the pipeline, deeply analyzes current benchmarking results, and delineates the ongoing challenges and future development plans for the remainder of the academic term.

Background

To comprehensively understand the necessity and the operational mechanics of the proposed translation pipeline, it is imperative to examine the architectural constraints of the target BCI hardware, the computational paradigms introduced by the RISC-V Vector extension, and the foundational principles of the constituent compiler technologies utilized in this research.

The fundamental engineering challenge in the development of chronic, implantable Brain-Computer Interfaces is the strict management of thermal dissipation. The human cerebral cortex is highly sensitive to temperature fluctuations, and prolonged exposure to elevated temperatures can induce irreversible tissue damage and neuroinflammation.[3] Consequently, international medical standards dictate that an implanted device must not raise the local tissue temperature by more than one degree Celsius, effectively restricting the total power consumption of the entire BCI system—including amplifiers, analog-to-digital converters, wireless telemetry, and digital signal processing units—to roughly fifteen milliwatts.[3]

Historically, BCI designers have relied on custom-designed ASICs to meet these stringent power requirements. While ASICs offer unparalleled energy efficiency for highly specific computational kernels, they inherently lack flexibility. If a neuroscientist develops a superior seizure-prediction algorithm or wishes to repurpose the implant for a different neurological disorder, the ASIC cannot be updated; a completely new physical chip must be engineered, fabricated, and surgically implanted.[2] General-purpose microprocessors offer the requisite flexibility through software updates but typically consume power orders of magnitude beyond the safe physiological limit.[3]

The HALO project resolves this historical dichotomy through aggressive hardware-software co-design. The architecture is built around a configurable interconnect that links a heterogeneous array of application-specific PEs.[2] These PEs are meticulously optimized for the specific computational kernels most frequently encountered in neural signal processing, such as Lempel-Ziv compression algorithms, Non-Linear Energy Operators for neuronal spike detection, and Butterworth Bandpass Filters for isolating specific brain wave frequencies.[3] These accelerators can be dynamically chained together to form highly efficient data flow pipelines tailored to specific clinical tasks.[2]

To manage these pipelines and execute computational tasks that fall outside the purview of the dedicated hardware accelerators, HALO incorporates a low-power RISC-V microcontroller.[2] To ensure that this software-driven execution does not violate the power budget, the microcontroller relies heavily on the RISC-V Vector extension.[1] Unlike traditional Single

Instruction, Multiple Data architectures, which utilize fixed-width vector registers and require constant recompilation for different hardware generations, RVV introduces a vector-length agnostic programming model.[10]

In the context of BCI workloads, which frequently involve matrix multiplications and convolutions over varying temporal window sizes, RVV allows the processor to load large, contiguous segments of multi-channel neural data into vector registers and apply a single arithmetic instruction across the entire dataset simultaneously.[1] This architectural paradigm significantly reduces the dynamic power consumed by the processor's instruction fetch and decode units. By replacing hundreds or thousands of scalar loop iterations with a single vector instruction, the processor minimizes the total number of active clock cycles required to complete a given task.[1] Consequently, computational algorithms are executed much faster, allowing the microcontroller to aggressively utilize low-power sleep states between neural data sampling intervals—a crucial mechanism for extending the operational lifespan of the implantable battery.[1]

Despite these sophisticated hardware mechanisms, the platform remains inaccessible to the primary end-users: neuroscientists. Clinical algorithm development relies almost exclusively on high-level, dynamically typed languages like MATLAB, which abstract away memory management and hardware-specific optimizations.[1] The automated pipeline proposed in this research bridges this gap by integrating two existing, albeit incomplete, software repositories. The first is a TypeScript-based source-to-source compiler designated as ts-traversal, which is tasked with parsing MATLAB syntax, inferring static types, and generating functional C code.[1] The second is a C library designated as RISCV-Matrix, which maps standard mathematical operations directly to RVV assembly instructions.[1] While the conceptual foundation of these tools is sound, critical gaps remain in both the translation logic and the vectorization coverage, necessitating extensive debugging, architectural integration, and rigorous validation against the target BCI workloads.

Design, Platform, and Modeling

The core objective of this research is the architecting and implementation of a cohesive, automated compilation pipeline capable of translating high-level algorithmic intent into hardware-optimized silicon instructions. This section details the system architecture, focusing on the sequential transformation mechanisms, the categorization of vectorizable workloads, and the underlying platform dependencies utilized for modeling and validation.

The implementation of the compiler pipeline is designed as a continuous, directed data flow that meticulously isolates the high-level semantic analysis from the low-level hardware optimizations. The system architecture can be visualized through a structured sequence of state transitions and module interactions. The pipeline initiates with the ingestion of raw .m files authored by clinical researchers. These files contain standard MATLAB syntax, incorporating dynamic array allocations, calls to proprietary signal processing toolboxes, and high-level,

implicitly parallel matrix arithmetic.

These raw files are passed into the first major component of the pipeline, the ts-traversal engine.[1] This engine executes a multi-phase transformation. First, it utilizes the tree-sitter and tree-sitter-matlab parsing libraries to generate a Concrete Syntax Tree.[1] This hierarchical representation captures the complete syntactic structure of the input algorithms. Because MATLAB is a dynamically typed language—allowing variables to transition freely between scalar integers, complex floating-point numbers, and multi-dimensional matrices—the engine must perform rigorous type inference. The inference algorithms traverse the Concrete Syntax Tree to deduce the maximal dimensions, memory allocation requirements, and fundamental data types of all variables, mapping them permanently to statically typed C constructs.[1] The output of this initial phase is standard C code. At this juncture, the semantic intent of the MATLAB script is entirely preserved, but the resulting code remains architecturally agnostic and relies on standard, scalar loop iterations.

To introduce the necessary hardware optimizations, the pipeline employs an internal routing mechanism. The generated C code inherently consists of two distinct computational subsets. The non-vectorizable subset comprises complex control flow logic, sequential input/output operations, and highly divergent branching structures. Conversely, the vectorizable subset encompasses large-scale loops, dense matrix arithmetic, and parallelizable signal transformations. The routing mechanism isolates this vectorizable subset and directs it into the second major component of the pipeline, the RISCV-Matrix library.

Within the RISCV-Matrix module, the standard scalar loops generated by the previous step are systematically replaced with specific Application Programming Interface calls mapped to the library's headers. These headers contain highly optimized, inline RISC-V Vector assembly intrinsics.[1] This transformation effectively converts the vectorizable subset into heavily accelerated C code explicitly tuned for the parallel execution capabilities of the HALO microcontroller.[1]

The final phase of the architectural model involves the compilation and deployment of the bifurcated code base. Both the unmodified non-vectorizable standard C code and the heavily vectorized C code are passed simultaneously into the RISC-V GNU Compiler toolchain. The compiler merges these subsets, statically linking the optimized vector library to produce a highly efficient RISC-V binary. This executable represents the culmination of the pipeline, containing the exact neuroscientific algorithms originally authored in MATLAB, now fully capable of meeting the stringent power and latency requirements of the implanted chip.

Because the physical HALO silicon is subject to iterative foundry tape-outs and restricted availability, the validation, execution modeling, and cycle-accurate benchmarking of the generated binaries are conducted using the Spike instruction set architecture simulator.[1] Spike provides a fully functional, cycle-accurate model of a RISC-V core equipped with the RVV extension, allowing for precise tracking of executed instructions, vector register utilization,

memory access latency, and overall execution cycles.[12] This platform enables the continuous, iterative refinement of both the source-to-source compiler and the vectorization library without requiring constant access to physical test hardware. To bridge the TypeScript translation engine and the C-based vectorization library, a unified build system is currently under development, designed to automate the entire data flow from source ingestion to simulator execution via a single command-line interface.[1]

Results

Extensive benchmarking and validation protocols have been executed during the first half of the research timeline, yielding highly granular insights into the performance characteristics of the generated binaries, the efficiency of the underlying vectorization library, and the specific failure modes of the translation engine. The primary focus has been the successful replication of the RISCV-Matrix library functionality, the comparative analysis of execution cycles across different toolchain iterations, the isolation of test-harness anomalies, and the active debugging of critical numerical algorithms.

The initial validation efforts confirmed that the RISCV-Matrix library, alongside the Spike RISC-V ISA simulator, compiles and executes correctly within the unified testing environment.[1] Foundational functionalities, including element-wise arithmetic, scalar broadcasting, and basic linear algebra subroutines, were successfully executed on the simulator. The execution logs verified that the inline assembly intrinsics were correctly interpreted by the simulator's vector implementation, establishing a reliable baseline for subsequent performance benchmarking.

To precisely quantify the computational efficiency gained through the pipeline, comparative benchmarks were executed using a comprehensive suite of matrix operations. Performance is measured in Giga-cycles, representing the total number of simulated clock cycles required by the processor to complete the workload. In the context of the fifteen-milliwatt HALO power budget, a reduction in the total cycle count directly correlates to a proportional reduction in active power consumption. The analysis compares a historical performance baseline, generated using an older cross-compiler toolchain and test suite configuration, against the current, optimized pipeline.

Table 1 details the total runtime comparison across twenty-eight distinct operational tests, highlighting the specific cycle counts and the corresponding percentage changes in execution speed.

Table 1: RISCV-Matrix Total Runtime Performance Comparison (Gcycles)

Operation Test	Old Runtime (Gcycles)	New Runtime (Gcycles)	Percentage Change

abs	9.86	9.25	-6.2%
and	2.22	1.91	-14.2%
arg	6.48	5.33	-17.7%
ceil	4.96	4.14	-16.5%
divide	5.47	5.64	+3.0%
equal	4.75	4.31	-9.3%
floor	4.97	4.15	-16.4%
full_duplicate	4.32	3.48	-19.4%
isEqual	3.79	3.14	-17.2%
matlab_ge	4.37	3.65	-16.6%
matlab_gt	4.36	3.63	-16.6%

matlab_le	4.36	3.64	-16.5%
matlab_lt	4.34	3.63	-16.5%
minus	4.86	0.95	-80.4%
mtimes	3.07	3.01	-2.2%
ne	4.81	4.36	-9.2%
not	1.51	1.28	-15.2%
octave_ge	51.90	51.60	-0.6%
octave_gt	51.90	51.60	-0.6%
octave_le	51.94	51.67	-0.5%
octave_lt	51.94	51.67	-0.5%
ones	0.05	0.03	-44.3%
or	2.15	1.93	-10.1%

pairwise_max	54.36	53.60	-1.4%
pairwise_min	54.42	53.67	-1.4%
plus	4.87	0.95	-80.5%
round	4.95	4.05	-18.2%
scale	1.65	0.22	-86.5%
times	5.05	0.95	-81.2%
true	1.54	1.35	-12.4%
TOTAL	**415.20**	**393.73**	**-5.2%**

An exhaustive analysis of the data presented in Table 1 reveals several critical insights regarding the current state of the compilation pipeline. Across the entire test suite, the overall execution time decreased by 5.2 percent, dropping from 415.20 Giga-cycles to 393.73 Giga-cycles. However, the distribution of these performance gains is highly non-uniform. The majority of the standard arithmetic and logical operations, such as ceiling calculations, logical ANDs, and greater-than-or-equal comparisons, experienced moderate speedups ranging from ten to twenty percent. This systemic improvement is directly attributable to significant enhancements within the underlying RISC-V GNU cross-compiler toolchain. The newer iterations of the GCC compiler generate substantially tighter scalar and vector code, featuring optimized register allocation algorithms and superior instruction scheduling that were unavailable during the generation of the historical baseline.

More prominently, four specific tests—minus, plus, times, and scale—exhibited dramatic execution time reductions exceeding eighty percent. A deep architectural review of the testing

methodology revealed that these exceptional figures are not indicative of an eighty percent enhancement in the vectorization logic itself. Instead, they are artifacts resulting from the removal of input/output overhead and a reduction in test scope. In the updated pipeline, the speed tests for these specific functions were stripped down to exclusively output total runtime metrics, drastically reducing the total number of type combinations evaluated per run. Therefore, the eighty percent reduction represents a decrease in testing volume rather than a pure algorithmic speedup. Excluding these four highly skewed artifacts, the genuine, toolchain-driven performance enhancement across the library remains consistent at approximately five percent.

Further analysis was conducted to evaluate the efficacy of the vectorization implementation relative to scalar execution across different variable types. Table 2 presents the vector-to-reference execution ratios at the largest tested matrix size, encompassing approximately one million elements. A ratio less than 1.0 indicates that the vectorized implementation is faster than the scalar reference, while a ratio greater than 1.0 indicates a performance degradation.

Table 2: Vector-to-Reference Execution Ratios at Largest Matrix Size ($N \approx 1M$)

Operation	**Int x Int**	**Double x Double**	**Complex x Complex**
and	0.638	0.493	0.365
divide	0.280	0.325	0.235
equal	0.173	0.191	0.161
isEqual	0.671	0.797	**1.172**
matlab_ge	0.282	0.253	0.254
matlab_gt	0.282	0.253	0.254

matlab_le	0.282	0.277	0.277
matlab_lt	0.282	0.277	0.277
ne	0.163	0.180	0.153
octave_ge	0.282	0.253	0.142
octave_gt	0.282	0.253	0.142
octave_le	0.282	0.277	0.141
octave_lt	0.282	0.277	0.141
or	0.648	0.518	0.462
pairwise_max	0.001	0.004	0.135
pairwise_min	0.001	0.004	0.134

The vector-to-reference ratios demonstrate overwhelming success for the RVV implementations across nearly all operational domains. The fastest recorded operations are the pairwise_max and pairwise_min functions applied to integer arrays, which executed at a ratio of 0.001, representing a thousand-fold speedup over standard scalar iteration. This extreme efficiency is a direct consequence of the RVV architecture's ability to seamlessly ingest massive swaths of continuous memory and execute single-cycle comparisons. Unary operations, such as absolute value calculations on complex numbers, similarly demonstrated exceptional

performance with a ratio of 0.004.

However, the raw data presented in Table 2 contains a critical anomaly that prompted an extensive debugging investigation. The isEqual test, when evaluating complex-by-complex matrices, returned a ratio of 1.172, indicating that the explicitly vectorized implementation was approximately seventeen percent slower than the unoptimized scalar reference. Furthermore, additional debugging output indicated that the arg function for complex numbers returned a precise ratio of 1.000, denoting absolute performance parity with no vectorization benefits realized.

An exhaustive review of the raw execution cycle data for the isEqual anomaly revealed a fundamental flaw in the test harness architecture rather than a failure of the RVV implementation. The diagnostic logs demonstrated that across all complex permutations that yielded a ratio greater than 1.0 (including integer-by-complex, double-by-complex, and complex-by-integer combinations), the total execution time remained locked at a constant 75 cycles for the vectorized version and 64 cycles for the reference version. Crucially, these cycle counts remained completely static regardless of the underlying matrix size, exhibiting identical latencies whether processing a single element or a matrix containing 1,050,625 elements.

This static cycle count indicates that neither the vectorized nor the scalar reference versions were actually iterating through the data payload. A forensic examination of the test data generation scripts revealed that the isEqual test routinely compared matrices populated with entirely divergent data streams; for instance, attempting to verify equality between a matrix populated entirely by ones and an alternating matrix populated by the function $(-1)^i \times (i+1)$. Because the very first element of the matrices differed, both the scalar and vectorized implementations encountered an immediate short-circuit condition, triggering an early-exit break or returning false on the initial comparison.

The discrepancy in performance—the reason the vectorized version was seventeen percent slower during this immediate failure—is directly attributable to vector setup overhead. Before a RISC-V processor can execute a vector instruction, it must process several scalar configuration instructions, most notably the vsetvl command, which configures the vector length and operational parameters. Furthermore, loading complex numbers requires specific stride-loading instructions. When the algorithm aborts after checking only a single element, the proportional cost of this initial vector setup overhead heavily outweighs the cost of the actual comparison, rendering it slower than a simple scalar memory load and compare sequence. However, in realistic neuroscientific workloads where the matrices are genuinely equal, or where equality must be verified across a full traversal of the dataset, the vectorized implementation vastly outperforms the scalar reference. This is empirically proven by the single test combination that successfully traversed the data (complex-by-double), which achieved a highly efficient vector-to-reference ratio of 0.015.

The secondary anomaly concerning the arg function ratio of 1.000 was similarly diagnosed. A

review of the compiled assembly confirmed that the explicitly vectorized C function merely acts as a wrapper around a standard scalar loop executing the atan2 calculation. Because the underlying complex argument calculation was never actually mapped to RVV intrinsics within the RISCV-Matrix library, no parallelization occurred, resulting in identical execution times.

Beyond the resolution of benchmarking anomalies, significant engineering resources have been allocated to addressing fundamental algorithmic failures within the ts-traversal compiler phase. The most critical work-in-progress involves the computation of the matrix determinant.[1] In high-level environments like MATLAB, the determinant function is executed via highly optimized, numerically stable linear algebra subroutines that typically employ Lower-Upper decomposition coupled with partial pivoting. However, the current semantic mapping within ts-traversal attempts to naively translate the MATLAB determinant operation into a recursive cofactor expansion in C. For any matrix exceeding trivial dimensions, this recursive implementation induces an exponential time complexity and rapidly exhausts the constrained stack memory of the embedded RISC-V target, resulting in catastrophic system failure. The ongoing debugging strategy involves bypassing this recursive translation entirely and mapping the determinant operation directly to a newly authored, strictly vectorized Lower-Upper decomposition algorithm within the hardware library, ensuring both numerical stability and strict adherence to embedded memory constraints.

Related Work

The development of automated compiler toolchains and highly specialized hardware architectures for neural signal processing represents a rapidly evolving niche within the broader discipline of embedded systems research. The architectural methodologies and compilation strategies employed in this project build upon, and differentiate themselves from, several significant efforts detailed in contemporary literature.

The foundational paradigm driving this research is heavily informed by previous explorations into hardware-software co-design for implantable medical devices. Karageorgos et al. provided the foundational proofs demonstrating that general-purpose, monolithic ASICs are inherently incapable of sustaining the thermal and power requirements demanded by modern, multi-modal Brain-Computer Interfaces. Their research advocated for the transition toward the heterogeneous array of PEs combined with a low-power RISC-V microcontroller, directly inspiring the HALO architecture utilized as the target platform for this project.[2] Similarly, subsequent research expanded upon these principles by proposing the SCALO architecture, which utilizes a distributed network of wireless brain implants.[8] The SCALO system successfully leverages accelerator-rich compute nodes that balance aggressive hardware-software co-design to integrate processing, memory storage, and wireless networking within profound power constraints.[8] While both the HALO and SCALO architectures successfully address the physical hardware limitations of neuro-technological implants, they do not resolve the software usability gap, underscoring the necessity of the automated MATLAB-to-RVV translation

pipeline developed in this work.

Further research into real-time processing platforms for mobile BCIs, such as the FPGA-based CereBridge system and the MasterMind architecture, highlights the growing industry consensus regarding the decomposition of complex tasks into dedicated hardware accelerators.[15] These platforms demonstrate that directly routing data transfers between specific processing nodes significantly reduces the energetic burden placed on the central processor and main memory.[15] However, as with the HALO platform, the programming of these specialized data flows remains highly complex and largely inaccessible to researchers without deep expertise in register-transfer level design or embedded C programming.

In the domain of high-level synthesis and automated code generation, commercial tools such as MathWorks' MATLAB Coder provide a highly refined reference point for source-to-source translation.[1] MATLAB Coder is profoundly proficient in parsing proprietary MATLAB scripts and generating optimized, multi-core OpenMP C and C++ code targeted primarily at advanced desktop processors and commercial ARM Cortex architectures. However, these commercial synthesis tools are structurally agnostic regarding emerging, open-source instruction sets; they entirely lack support for generating RISC-V Vector intrinsics or interfacing with custom hardware accelerators.[1] Consequently, utilizing existing commercial compilation tools for the HALO platform yields unoptimized, scalar code that uniformly fails to meet the stringent fifteen-milliwatt power targets necessary for clinical deployment.

In a parallel context, Python—another dynamically typed language widely utilized in BCI data analysis and signal processing—has seen similar optimization efforts through tools like Cython. Cython addresses the performance bottlenecks of vanilla Python by allowing developers to compile Python code directly into efficient C or C++ extensions, utilizing proper type definitions to achieve substantial execution speedups. While this transition from a high-level language to compiled C code is highly effective for accelerating data pipelines, enabling true multiprocessing, and facilitating real-time processing on standard architectures, it shares the same critical limitation as MATLAB Coder in the context of this research: it does not inherently support the automated generation of targeted RISC-V Vector intrinsics or provide direct interfaces with specialized, ultra-low-power BCI hardware accelerators.

Finally, the vectorization of complex spectral analyses remains a pervasive challenge across all high-performance computing domains. While standard linear algebra routines have well-documented and highly refined optimization strategies, mapping algorithms like the Fast Fourier Transform to novel vector architectures is notoriously complex. Current implementations within the RISCV-Matrix library heavily rely on the legacy FFTW framework, which currently executes strictly in scalar mode because the underlying twiddle factor calculations and butterfly operations have not yet been successfully mapped to RVV intrinsics.[1] Recent independent research into accelerating Number Theoretic Transforms and Fast Fourier

Transforms on domain-specific RISC-V cores has demonstrated that when these algorithms are properly tensorized and mapped to vector intrinsics, execution throughput can increase by a factor of eighteen.[9] These findings validate the core objective of this project to aggressively pursue the implementation of proper FFT vectorization into the RISCV-Matrix library during the latter half of the research term.

Conclusion and Future Plan

The successful chronic implantation of next-generation Brain-Computer Interfaces is entirely dependent on extreme power efficiency—a constraint that has historically been met through the deployment of highly inflexible, monolithic hardware. The Hardware Architecture for LOw Power BCIs leverages a heterogeneous processing array and the RISC-V Vector extension to introduce unprecedented flexibility without compromising critical thermal limits. However, to make this sophisticated hardware accessible to the broader neuroscientific community, an automated, highly optimized software compilation pipeline is absolutely required. This project directly addresses this need by engineering an end-to-end toolchain that merges a TypeScript-based source-to-source compiler with a hardware-specific vectorization library, enabling the automatic transformation of high-level MATLAB algorithms into power-optimized RISC-V binaries.

Preliminary validation and extensive benchmarking demonstrate that the underlying vectorization library successfully executes within the cycle-accurate Spike simulator environment, yielding consistent performance enhancements across fundamental vector operations compared to their scalar equivalents. Exhaustive analysis of the benchmarking data has successfully isolated performance improvements driven by updated cross-compiler toolchains, while simultaneously identifying and resolving significant test-harness anomalies—most notably the short-circuit evaluation artifacts that artificially degraded the apparent performance of vectorized equality checks. Despite these successes, critical challenges remain isolated within the semantic translation layer. Ongoing debugging efforts are actively addressing numerical instability and stack overflow issues associated with the recursive translation of matrix determinant computations, highlighting the need for robust, hardware-accelerated linear algebra subroutines.

The strategic roadmap for the remainder of the academic semester is meticulously structured to resolve these existing bugs, substantially expand the vectorization coverage of the hardware library, and achieve full end-to-end integration of the compilation pipeline.

During the immediate developmental phase spanning weeks seven through nine, the primary engineering focus will shift entirely toward expanding the RISCV-Matrix library to encompass critical, currently non-vectorized BCI functions. This effort includes implementing a numerically stable, RVV-optimized Lower-Upper decomposition algorithm to safely compute matrix determinants, thereby replacing the failing recursive translation logic. Furthermore, aggressive efforts will be directed toward vectorizing the Fast Fourier Transform and its associated

spectral windowing functions, alongside the introduction of heavily vectorized statistical extraction routines, including mean, standard deviation, and variance calculations.[1]

Following the expansion of the hardware library, weeks ten through twelve will be dedicated to complete system integration and rigorous testing. The currently independent translation and vectorization repositories will be unified under a single, cohesive build system. An automated testing harness will be engineered to seamlessly ingest standard neuroscientific MATLAB scripts, execute the resulting compiled RISC-V binaries via the Spike simulator, and mathematically compare the generated outputs against reference outputs produced natively within MATLAB. This rigorous testing protocol is essential to guarantee the deterministic correctness of the translated algorithms prior to physical silicon deployment.[1]

Finally, the concluding phase spanning weeks thirteen through sixteen will focus heavily on end-to-end benchmarking and documentation. The fully integrated pipeline will be subjected to realistic, full-scale BCI signal processing workloads, such as complete seizure onset detection algorithms. Comprehensive performance benchmarking will be conducted to definitively quantify total power savings and latency reductions achieved through the automated vectorization process. The project will culminate in the production of a comprehensive final senior thesis report and a formal poster presentation detailing the architecture and capabilities of the unified framework.[1] By successfully bridging the semantic gap between high-level neuroscience research tools and ultra-low-power embedded hardware, this compilation pipeline will significantly accelerate the development and clinical deployment of next-generation neurological therapies.

Works cited

1. ignore
2. Hardware-Software Co-Design for Brain-Computer Interfaces, accessed March 2, 2026, https://www.cs.yale.edu/homes/abhishek/ksriram-isca20.pdf
3. HALO - UNC Computer Science, accessed March 2, 2026, https://www.cs.unc.edu/~raghav/Documents/microhc23.pdf
4. HALO: A Hardware-Software Co-Designed Processor for Brain-Computer Interfaces, accessed March 2, 2026, https://csl.yale.edu/~rajit/ps/MM3258907.pdf
5. Computer Interfaces - NSF Public Access Repository, accessed March 2, 2026, https://par.nsf.gov/servlets/purl/10310810
6. HALO | Yale Ventures, accessed March 2, 2026, https://ventures.yale.edu/node/1886
7. RETROSPECTIVE: Hardware-Software Co-Design for Brain-Computer Interfaces - Yale University, accessed March 2, 2026, https://csl.yale.edu/~rajit/ps/HALO_Retrospective.pdf
8. Hardware-Software Co-Design for Brain-Computer Interfaces - ResearchGate, accessed March 2, 2026, https://www.researchgate.net/publication/342911598_Hardware-Software_Co-Design_for_Brain-Computer_Interfaces

9. Programme | DATE 2024, accessed March 2, 2026, https://date24.date-conference.com/programme
10. Università Degli Studi Di Napoli Federico II - fedOA, accessed March 2, 2026, http://www.fedoa.unina.it/13259/1/PhDThesisPipicelliFinal.pdf
11. Rajit Manohar - Computer Systems Lab @ Yale, accessed March 2, 2026, https://csl.yale.edu/~rajit/cv.pdf
12. riscv-stc/riscv-matrix-project: Top project for RISC-V Matrix extension proposal and related opensource implementations. - GitHub, accessed March 2, 2026, https://github.com/riscv-stc/riscv-matrix-project
13. Hardware-Software Co-Design for Brain-Computer Interfaces - IEEE Xplore, accessed March 2, 2026, https://ieeexplore.ieee.org/document/9138938/
14. SCALO: An Accelerator-Rich Distributed System for Scalable Brain-Computer Interfacing, accessed March 2, 2026, https://www.cs.yale.edu/homes/abhishek/ksriram-isca23.pdf
15. MasterMind: Many-Accelerator SoC Architecture for Real-Time Brain-Computer Interfaces, accessed March 2, 2026, http://www.cs.columbia.edu/~davide_giri/pdf/eichler_iccd21.pdf
16. ISBN # 1-60132-523-1; American Council on Science & Education / CSCE 2025, accessed March 2, 2026, https://american-cse.org/static/CSCE25-book-abstracts.pdf